Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Data Structure Basics

C Programming

C is a procedural, mid-level programming language known for its efficiency, portability, and close-to-hardware control. It follows a structured, function-based approach, making it ideal for system programming (OS kernels, drivers) and embedded systems. C is fast due to low-level memory access (pointers) and static typing, requiring explicit variable declarations. It lacks built-in OOP features and garbage collection, relying on manual memory management (malloc/free). Despite being old, C remains widely used due to its portability (runs on almost any system) and rich standard library (stdio.h, math.h). Its case-sensitive syntax and modularity (header files) make it flexible yet demanding precise coding.

Properties of C Programming

1. Procedural Paradigm

Follows a linear, function-driven approach (unlike OOP). Data Structures are implemented using functions (e.g., push(), pop() for stacks)

Example:

```
struct Stack {
 int top;
 int items[MAX];
 };
void push(struct Stack* s, int x) {
 /*...*/
 }
```

2. Static Typing

Variables require explicit type declaration. This enforces type safety in DS implementations.

Example:

```
int arr[10];        // Fixed-type array
float* dynamicArr; // Type-specific pointer
```

3. Memory Management

Manual allocation/deallocation via malloc(), calloc(), free().

Example:

```
struct Node* newNode = (struct Node*)malloc(sizeof(struct Node));
```

```
free(newNode);
```

4. Pointer Support

Direct memory access enables efficient DS operations.
It is used in linked structures (nodes with next pointers) and pass-by-reference function arguments.

Example:

```
void insertNode(struct Node** head_ref) {
 /*...*/
 }
```

5. Efficiency

- Compiles to optimized machine code
- Enables high-performance DS implementations
- Used in time-critical applications (searching/sorting)

6. Portability

- Code can run across platforms
- Standard libraries ensure consistent behavior

7. Rich Standard Library

Essential DS-building components:

- stdio.h (I/O operations)
- stdlib.h (memory management)
- string.h (array manipulations)

8. Lack of Built-in DS

- No native implementations of stacks/queues
- Requires manual implementation
- This provides complete control over functionality.

Introduction to Data Structure

A data structure is a way to store and organize data in a computer so that it can be accessed and modified efficiently. It determines how data is arranged in memory and defines the operations (like insert, delete, search) that can be performed on it.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Importance of Data Structures

- **Efficiency:** Helps in optimizing time (speed) and space (memory).
- **Real-world Applications:** Used in databases, operating systems, AI, gaming, and more.
- **Problem-solving:** Essential for writing optimized algorithms.

Basic Types

1. Primitive (Built-in): Basic data types like int, float, char.
2. Non-Primitive (Derived)
    a. Linear: Arrays, Linked Lists, Stacks, Queues.
    b. Non-Linear: Trees, Graphs.

Key Operations performed on a Data Structure

- Insertion – Adding data
- Deletion – Removing data
- Traversal – Accessing all data
- Searching – Finding specific data
- Sorting – Arranging data in order

Example:

Array: Stores multiple items of the same type.

```
int marks[5] = {90, 85, 78, 92, 88};
```

Linked List: Dynamically stores data using nodes.

```
struct Node {
    int data;
    struct Node* next;
};
```

Memory allocation

Memory allocation refers to the process of assigning space in a computer's memory to store data during program execution.

Static Memory Allocation

- Memory is allocated at compile time.
- Size is fixed and cannot be changed during runtime.
- Variables are stored in the stack memory.
- Example: Arrays with predefined size (int arr[10];).

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Dynamic Memory Allocation

- Memory is allocated at runtime using functions like malloc(), calloc(), or realloc().
- Size can be modified as needed.
- Variables are stored in the heap memory.
- Example: Linked lists, dynamically resized arrays (int *arr = malloc(n * sizeof(int));).

Searching

Searching refers to the process of finding a specific element in a data structure. Common methods include:

1. Linear Search → Checks each element one by one (Simple but slow for large data).
2. Binary Search → Finds elements by repeatedly dividing a sorted list in half (Fast but requires sorted data).

Sorting

Sorting arranges elements in a specific order (ascending/descending). Common algorithms:

1. Bubble Sort → Repeatedly swaps adjacent elements if they're in the wrong order (Easy but slow).
2. Selection Sort → Selects the smallest element and swaps it into place (Simple but inefficient).
3. Insertion Sort → Builds the sorted list one item at a time (Good for small/nearly sorted data).
4. Quick Sort → Divides data around a pivot and sorts recursively (Fastest for large datasets).
5. Merge Sort → Splits data, sorts halves, then merges them (Consistent speed, uses extra space).

Algorithm: Definition and Properties

An algorithm is a step-by-step procedure to solve a problem or perform a computation. It takes input data, processes it through a finite sequence of instructions, and produces the desired output.

Key Properties of an Algorithm

1. Input: Must accept zero or more externally supplied inputs.
2. Output: Must produce at least one meaningful result.
3. Finiteness: Must terminate after a finite number of steps (no infinite loops).
4. Definiteness: Each step must be clear, precise, and unambiguous.
5. Effectiveness: Every step must be executable without external intelligence (no vague instructions).

Example:

Algorithm to find the maximum number in an array:

Input: An array of numbers.
Steps:

1. Initialize max = first element.
2. Compare max with each remaining element.
3. If any element > max, update max.

Output: Return max.

Types of Algorithms

Algorithms can be broadly classified into iterative, recursive, divide-and-conquer, greedy, dynamic programming, and backtracking methods.

Iterative Algorithms

An iterative algorithm solves problems by repeating a set of instructions in a loop until a condition is met. Unlike recursive approaches (which call themselves), iterative methods use loops (e.g., for, while) to process data step-by-step, making them memory-efficient (no stack overhead) and often easier to debug. Examples include linear search (sequentially checking each element) and Bubble Sort (repeatedly swapping adjacent elements). Iterative algorithms are preferred for large datasets due to their lower space complexity and straightforward implementation.

Key Features:

- Uses loops for repetition
- No extra memory for call stacks
- Easier optimization for performance
- Example: Calculating factorial with a for loop instead of recursion.

Algorithm Analysis

When multiple algorithms exist to solve the same problem, analysis helps determine the most suitable one based on specific criteria. Key factors for evaluation include:

Metrics for Algorithm Analysis

1. Robustness

- Does the algorithm handle all valid and invalid inputs correctly?
- Example: A sorting algorithm should not crash with empty or duplicate inputs.

2. Maintainability

- Is the algorithm easy to modify or extend for future needs?
- Example: Well-structured code with comments aids updates.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

3. Scalability

- Can it efficiently process larger inputs without performance drops?
- Example: An $O(n^2)$ algorithm becomes slow for big data vs. $O(n \log n)$.

4. Modularity

- Can it be divided into reusable, independent components?
- Example: Breaking a search algorithm into "sort" and "binary search" modules.

5. Security

- Is it resistant to malicious inputs or attacks (e.g., buffer overflows)?
- Example: Input validation in cryptographic algorithms.

6. User-Friendliness

- Is the algorithm intuitive to implement and use?
- Example: Simple functions for sorting : sort().

7. Performance (Most Critical)

- Time Complexity: How fast does it run as input grows?
- Space Complexity: How much memory does it use?
- Example: QuickSort ($O(n \log n)$) outperforms BubbleSort ($O(n^2)$) for large datasets.

Need for Focus on Performance

1. Quantifiable: Time/space metrics provide objective comparisons.
2. Universal Impact: Directly affects real-world efficiency (e.g., server response times).
3. Bottleneck Identification: Highlights inefficiencies in large-scale systems.

Example:

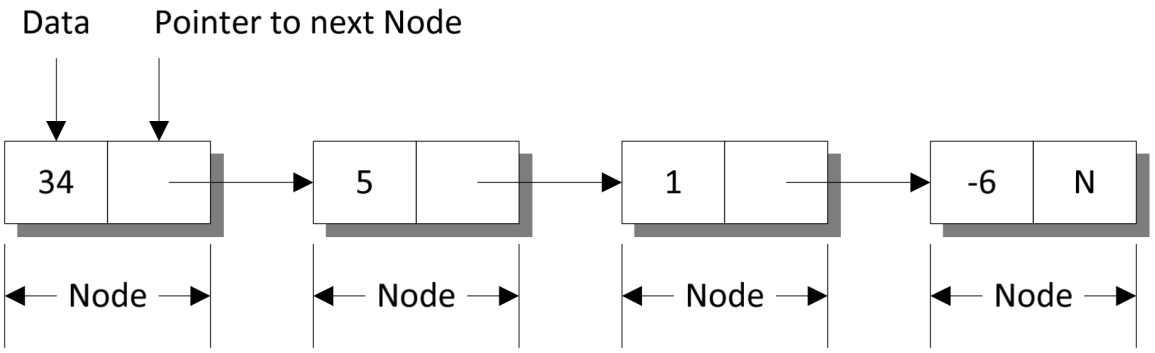
- Scenario: Choosing between Linear Search ($O(n)$) and Binary Search ($O(\log n)$):
  - Binary Search wins for performance (faster on sorted data).
  - Linear Search wins for maintainability (no need to sort first).

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Unit 1 - Arrays

Data structures are classified into two categories—linear and nonlinear. The elements in a linear data structure form a sequence, whereas elements in a nonlinear data structure do not. There are two ways of representing linear data structures in memory— Array based lists (simply called arrays) and Linked Lists. In an array the linear relationship between elements is established by storing its elements in sequential memory locations. In the linked list the linear relationship is established through pointers or links. In a linked list each node contains the data and the address of the next node.

(a) Array of 6 integers



(b) Linked list of 4 integers

Array - Definition & Basics

An array is a collection of similar data items stored in contiguous memory locations, allowing efficient access to elements using their index or position. Each element in the array can be directly accessed through its index, starting from zero. Arrays have a fixed size, meaning their length is defined at the time of declaration and cannot be changed during runtime, making them a static data structure.

Properties of Arrays

- Homogeneous: All elements are of the same data type.
- Contiguous Memory: Elements are stored in adjacent memory locations.
- Random Access: Elements can be accessed directly using index.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Declaration & Initialization

```
int arr[5]; // Declaration
int arr[5] = {1, 2, 3, 4, 5}; // Initialization
```

Operations on Arrays

Operation	Description
Traversal	Processing each element in the array
Search	Finding the location of an element with a given value
Insertion	Adding a new element to an array
Deletion	Removing an element from an array
Sorting	Organizing the array elements in some order
Merging	Combining two arrays into a single array
Reversing	Reversing the elements of an array

Single-Dimensional Arrays

- Elements are stored in a linear fashion.
- Base address = Address of the first element (arr[0]).
- Address of arr[i] = Base Address + (i * size_of_element)

Example Programs

1. Sum of Array Elements

```
int sum = 0;
for(int i = 0; i < n; i++)
    sum += arr[i];
```

2. Finding Maximum Element

```
int max = arr[0];
for(int i = 1; i < n; i++)
    if(arr[i] > max) max = arr[i];
```

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

3. Function to insert an element at a given position

```
void insert(int *arr, int pos, int num) {
    if (n >= MAX) {
        printf("Array is full. Cannot insert.\n");
        return;
    }
    if (pos < 0 || pos > n) {
        printf("Invalid position.\n");
        return;
    }
    for (int i = n; i > pos; i--) {
        arr[i] = arr[i - 1];
    }
    arr[pos] = num;
    n++;
}
```

4. Function to delete an element from a given position

```
void del(int *arr, int pos) {
    if (n == 0) {
        printf("Array is empty. Cannot delete.\n");
        return;
    }
    if (pos < 0 || pos >= n) {
        printf("Invalid position.\n");
        return;
    }
    for (int i = pos; i < n - 1; i++) {
        arr[i] = arr[i + 1];
    }
    arr[n - 1] = 0; // Optional: Clear last element
    n--;
}
```

5. Function to reverse the array

```
void reverse(int *arr) {
    int start = 0, end = n - 1, temp;
    while (start < end) {
```

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

```
        temp = arr[start];
        arr[start] = arr[end];
        arr[end] = temp;
        start++;
        end--;
    }
}
```

6. Function to display the array

```
void display(int *arr) {
    printf("Array: ");
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}
```

7. Function to search for an element in the array

```
void search(int *arr, int num) {
    for (int i = 0; i < n; i++) {
        if (arr[i] == num) {
            printf("Element %d found at position %d\n", num, i);
            return;
        }
    }
    printf("Element %d not found in array.\n", num);
}
```

Two-Dimensional Arrays

A 2-dimensional array is a collection of elements placed in m rows and n columns. The syntax used to declare a 2-D array includes two subscripts, of which one specifies the number of rows and the other specifies the number of columns of an array. These two subscripts are used to reference an element in an array. For example, arr[3][4] is a 2-D array containing 3 rows and 4 columns and arr[0][2] is an element placed at 0th row and 2nd column in the array. The two-dimensional array is also called a matrix.

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

COLUMN

ROW	0	1	2	3
0	12	1	-9	23
1	14	7	11	121
2	6	78	15	34

Row Major and Column Major Arrangement

Rows and columns of a matrix are only a matter of imagination. When a matrix gets stored in memory all its elements are stored linearly since computer memory can only be viewed as consecutive units of memory locations. This leads to two possible arrangements of elements in memory Row Major Arrangement and Column Major Arrangement.

```
int a[ 3 ][ 4 ] = {
                    { 12, 1, -9, 23 },
                    { 14, 7, 11, 121 },
                    { 6, 78, 15, 34 }
                } ;
```

Row Major Arrangement

0th row				1st row				2nd row			
12	1	-9	23	14	7	11	121	6	78	15	34
502	506	510	514	518	522	526	530	534	538	542	546

Column Major Arrangement

0th col			1st col			2nd col			3rd col		
12	14	6	1	7	78	-9	11	15	23	121	34
502	506	510	514	518	522	526	530	534	538	542	546

Note: Each integer occupies four-bytes

Row Major Arrangement

Element 121 is present at a[1][3]. Hence location of 121 would be

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

✅ **Row-Major Address Formula:**

$$\text{Address}(a[i][j]) = \text{Base Address} + ((i \times \text{Cols}) + j) \times \text{Element Size}$$

🔢 **Substitute the values:**

$$= 502 + ((1 \times 4) + 3) \times 4$$
$$= 502 + (4 + 3) \times 4$$
$$= 502 + 7 \times 4$$
$$= 502 + 28$$
$$= \boxed{530}$$

Column Major Arrangement

Element 121 is present at a[1][3]. Hence location of 121 would be

✅ **Column-Major Address Formula:**

$$\text{Address}(a[i][j]) = \text{Base Address} + ((j \times \text{Rows}) + i) \times \text{Element Size}$$

🔢 **Substitute values:**

Let's assume **rows = 3** (as the minimum possible value for `i = 1` to exist).

$$= 502 + ((3 \times 3) + 1) \times 4$$
$$= 502 + (9 + 1) \times 4$$
$$= 502 + 10 \times 4$$
$$= 502 + 40$$
$$= \boxed{542}$$

Algorithms for Common Matrix Operations

1. To Create (Input) a Matrix

```
void inputMatrix(int mat[10][10], int rows, int cols) {
    printf("Enter elements of %d x %d matrix:\n", rows, cols);
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            scanf("%d", &mat[i][j]);
```

```
        }
    }
}
```

2. To Display a Matrix

```
void displayMatrix(int mat[10][10], int rows, int cols) {
    printf("Matrix:\n");
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            printf("%d\t", mat[i][j]);
        }
        printf("\n");
    }
}
```

3. Matrix Addition

```
void addMatrix(int A[10][10], int B[10][10], int result[10][10],
int rows, int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            result[i][j] = A[i][j] + B[i][j];
        }
    }
}
```

Note: *Both matrices must be of same size.*

4. Matrix Multiplication

```
void multiplyMatrix(int A[10][10], int B[10][10], int
result[10][10], int r1, int c1, int c2) {
    for (int i = 0; i < r1; i++) {
        for (int j = 0; j < c2; j++) {
            result[i][j] = 0;
            for (int k = 0; k < c1; k++) {
                result[i][j] += A[i][k] * B[k][j];
            }
        }
    }
}
```

Note: *Matrix A (r1 × c1) and Matrix B (c1 × c2) => Result will be (r1 × c2)*

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

5. Transpose of a Matrix

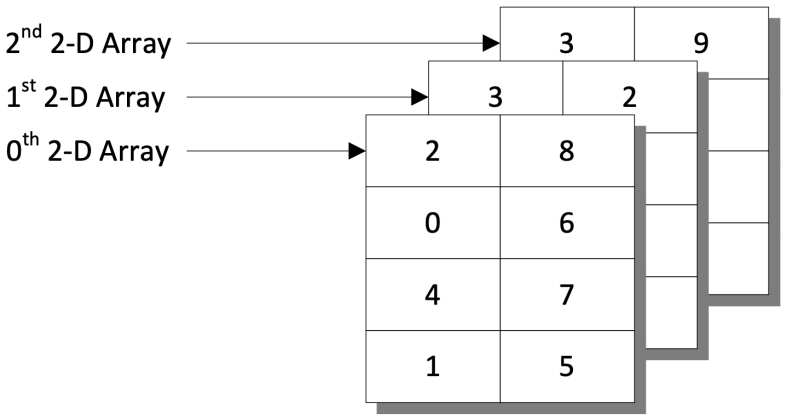
```
void transposeMatrix(int mat[10][10], int trans[10][10], int rows,
int cols) {
    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            trans[j][i] = mat[i][j];
        }
    }
}
```

Applications of Two-Dimensional Arrays

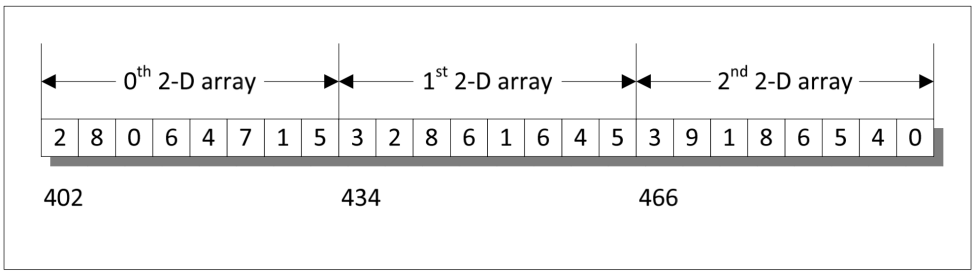
- Matrices in mathematics
- Game boards (Chess, Tic-Tac-Toe)
- Image processing (Pixels as 2D arrays)

Multi-Dimensional Arrays

- Arrays with more than one dimension (e.g., 2D, 3D).
- A 3-dimensional array can be thought of as an array of arrays of arrays.
- Figure below shows a 3-D array, which is a collection of three 2-D arrays each containing 4 rows and 2 columns.



The outer array has three elements, each of which is a 2D array, which in turn holds four 1D arrays containing two integers each. Note that the arrangement shown in the above figure is only conceptually true. In memory the same array elements are stored linearly as shown below.



Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Arrays and Polynomials

Polynomials like $5X^4 + 2X^3 + X^2 + 10X - 8$ can be maintained using an array. The simplest way to represent a polynomial of degree n is to store the coefficient of (n + 1) terms of a polynomial in an array. For this each element of the array should consist of two values—coefficient and exponent. While storing the polynomial it is assumed that the exponent of each successive term is less than that of the previous term. Once we build an array to represent a polynomial, we can use it to perform common polynomial operations like addition and multiplication.

Given Structures

```
struct term {
    int coeff;
    int exp;
};
struct poly {
    struct term t[10];
    int noofterms;
};
```

Each polynomial is represented as an array of terms (up to 10), where each term contains a coefficient and exponent.

Algorithms

1. Algorithm to Add Two Polynomials

Assume both p1 and p2 are sorted in descending order of exponents.

```
struct poly addPoly(struct poly p1, struct poly p2) {
    struct poly result;
    int i = 0, j = 0, k = 0;
    result.noofterms = 0;

    while (i < p1.noofterms && j < p2.noofterms) {
        if (p1.t[i].exp > p2.t[j].exp) {
            result.t[k++] = p1.t[i++];
        } else if (p1.t[i].exp < p2.t[j].exp) {
            result.t[k++] = p2.t[j++];
        } else {
            result.t[k].exp = p1.t[i].exp;
            result.t[k].coeff = p1.t[i].coeff + p2.t[j].coeff;
            i++; j++; k++;
        }
    }
```

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

```
    while (i < p1.noofterms) {
        result.t[k++] = p1.t[i++];
    }

    while (j < p2.noofterms) {
        result.t[k++] = p2.t[j++];
    }

    result.noofterms = k;
    return result;
}
```

2. Algorithm to Multiply Two Polynomials

We multiply each term of p1 with every term of p2 and then combine like terms.

```
struct poly multiplyPoly(struct poly p1, struct poly p2) {
    struct poly temp, result;
    temp.noofterms = 0;

    for (int i = 0; i < p1.noofterms; i++) {
        for (int j = 0; j < p2.noofterms; j++) {
            temp.t[temp.noofterms].exp = p1.t[i].exp +
p2.t[j].exp;
            temp.t[temp.noofterms].coeff = p1.t[i].coeff *
p2.t[j].coeff;
            temp.noofterms++;
        }
    }

    // Combine like terms in temp → result
    result.noofterms = 0;
    for (int i = 0; i < temp.noofterms; i++) {
        int found = 0;
        for (int j = 0; j < result.noofterms; j++) {
            if (temp.t[i].exp == result.t[j].exp) {
                result.t[j].coeff += temp.t[i].coeff;
                found = 1;
                break;
            }
        }
        if (!found) {
```

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

```
            result.t[result.noofterms] = temp.t[i];
            result.noofterms++;
        }
    }

    return result;
}
```

Sparse Matrices

A matrix where most elements are zero.Storing it as a 2D array is memory inefficient. If the matrix is sparse, we must consider an alternate way of representing it rather than the normal row major or column major arrangement. This is because if majority of elements of the matrix are 0 then an alternative through which we can store only the non-zero elements and keep intact the functionality of the matrix can save a lot of memory space.

Columns

Rows

	0	1	2	3	4	5	6
0	0	0	0	5	0	0	0
1	0	4	0	0	0	0	7
2	0	0	0	0	9	0	0
3	0	3	0	2	0	0	0
4	1	0	2	0	0	0	0
5	0	0	0	0	0	0	0
6	0	0	8	0	0	0	0

Array Representation (3-Tuple Format)

A sparse matrix is a matrix in which most of the elements are zero. To save space, we store only the non-zero elements along with their row and column indices.

Structure of 3-Tuple Representation

- ➔ The first row stores:
  [number of rows, number of columns, number of non-zero elements]
- ➔ Each of the following rows stores one non-zero element:
  [row index, column index, value]

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

Example

Original 3×3 Matrix:

```
0  0  3
0  0  0
0  2  0
```

3-Tuple Representation:

```
[3 3 2]
[0 2 3]
[2 1 2]
```

Algorithm - Array Representation of Sparse Matrix

```
void convertToSparse(int mat[10][10], int rows, int cols, int
sparse[100][3]) {
    int k = 1;  // Index for sparse matrix (starts from 1)

    sparse[0][0] = rows;
    sparse[0][1] = cols;
    sparse[0][2] = 0;  // To be updated with count of non-zero
elements

    for (int i = 0; i < rows; i++) {
        for (int j = 0; j < cols; j++) {
            if (mat[i][j] != 0) {
                sparse[k][0] = i;
                sparse[k][1] = j;
                sparse[k][2] = mat[i][j];
                k++;
            }
        }
    }

    sparse[0][2] = k - 1;  // Store count of non-zero elements
}
```

Advantages of a Sparse Matrix

- Saves memory for large sparse matrices.
- Faster operations (addition, multiplication).

Sparse Matrices (Linked Representation)

Each non-zero element is stored as a node with:

Ankita Kumari
Assistant Professor
Dept. of CA (Ranchi Women's College)

- Row, Column, Value
- Pointer to the next node

Structure

```
struct Node {
    int row, col, value;
    struct Node *next;
};
```

Advantages over Array Representation

- Dynamic memory allocation (no fixed size).
- Efficient insertion/deletion of non-zero elements.

Applications

- Graph representations (Adjacency List)
- Large scientific computations